Impulsa Tech, C.A.
IMPULSANDO EL FUTURO FINANCIERO

BITÁCORA DE IA

Entrada 1 - Arquitectura base del MVP

Fecha: 04/11/2026 **Entregable:** Base para todos los entregables

Prompt del equipo:

Hola! Te paso el PDF del proyecto Ahorra+ y los diagramas C4 de contexto y contenedores. Necesitamos armar el MVP como monolito modular con arquitectura hexagonal (puertos y adaptadores), backend en Java con Spring Boot, app en Flutter y PostgreSQL. ¿Me ayudas a organizar los módulos y las carpetas para que respete el C4? Explícamelo sencillo porque es la primera vez que hacemos hexagonal.

Qué hizo la IA: Propuso un módulo por cada contenedor del C4 (identidad, consolidación, cofre, pagos, puntos, perfil IA, notificaciones, backoffice) con capas dominio / aplicación / adaptador, y redactó docs/arquitectura.md con la trazabilidad C4 a código y las reglas del modelo E-R (R-01 a R-23).

Qué revisamos o cambiamos: Comparamos los módulos contra nuestro diagrama E-R de Lucid y renombramos algunas tablas para que coincidieran. Decidimos usar Spring Modulith para verificar que un módulo no entre en las tripas de otro.

Entrada 2 - Backend Merchant y servicios simulados

Fecha: 04/11/2026 **Entregable:** Prototipo backend (sandbox / suiche simulado)

Prompt del equipo:

No tenemos acceso a bancos reales ni al suiche, así que necesitamos mocks. ¿Puedes crear en una carpeta aparte unos simuladores del banco (con OAuth2 tipo open banking), del suiche interbancario, de la tasa BCV y del KYC? Que el suiche a veces apruebe, a veces rechace y a veces se tarde, para poder probar casos feos.

Qué hizo la IA: Creó mocks/simuladores/ como un servicio Spring Boot independiente (puerto 8090) con consola web para ver OTP y mensajes del suiche, escenarios configurables (aprobado, rechazado, timeout, diferencia en conciliación) y el contrato documentado en docs/contrato-simuladores.md.

Qué revisamos o cambiamos: Probamos cada escenario con scripts/prueba_e2e.py. Pedimos que los montos se manejaran en céntimos enteros (sin decimales flotantes) después de ver un error de redondeo en una prueba.

Entrada 3 - Prototipo front-end navegable

Fecha: 05/11/2026 **Entregable:** Prototipo front-end del flujo principal

Prompt del equipo:

Ahora la app en Flutter. El flujo principal que queremos mostrar es: registrarse, hacer el KYC, vincular un banco, ver el saldo consolidado en Bs y USD, apartar al cofre y pagar la tarjeta de crédito. ¿Se puede hacer que también funcione sin backend para la demo en clase?

Qué hizo la IA: Construyó la app en mobile/ahorra_mas_app con Riverpod y go_router, pantallas por feature y un modo FAKE_API que simula las respuestas del núcleo para presentar sin servidores.

Qué revisamos o cambiamos: Lo probamos en el emulador Android y nos tocó ajustar la URL (10.0.2.2) para que el banco simulado abriera en el navegador del teléfono. Luego pedimos un rediseño visual porque la primera versión se veía muy genérica.

Entrada 4 - Módulo Cofre Protector

Fecha: 05/11/2026 **Entregable:** Funcionalidad diferencial del flujo principal

Prompt del equipo:

Queremos agregar el módulo Cofre Protector con 4 cosas: la ruta del día (convertir la meta en una cuota diaria con la tasa BCV), la racha con días colchón, el Espejo de Impacto y el sello de compromiso. Importante: no podemos tocar el dinero del usuario, todo es lógico. ¿Cómo lo hacemos sin romper lo que ya está?

Qué hizo la IA: Agregó el cálculo de la cuota diaria, la racha que no se rompe con días colchón, el Espejo de Impacto (solo lectura del ledger y la tasa BCV, con aviso de “simulación informativa, no asesoría”) y el sello que solo quita el multiplicador de puntos si se rompe.

Qué revisamos o cambiamos: Revisamos que el sello nunca generara un movimiento en el ledger y agregamos el disclaimer en la pantalla para no meternos en terreno de SUDEBAN.

Entrada 5 - Diagramas de secuencia

Fecha: 05/11/2026 **Entregable:** Secuencias de autorización, captura, reverso y conciliación

Prompt del equipo:

El profe pide diagramas de secuencia de autorización, captura, reverso y conciliación, mostrando qué hace el suiche y cada actor. ¿Nos ayudas a sacarlos a partir de cómo quedó implementado el pago de TDC? Que se entienda quién llama a quién y en qué momento cambia el estado del pago.

Qué hizo la IA: Describió las cuatro secuencias con los actores (usuario, app, núcleo Ahorra+, banco emisor, suiche, banco de la tarjeta) y los estados del pago (AUTORIZADO, CAPTURADO, REVERSADO, CONCILIADO, CON_DIFERENCIA), basándose en el historial de estados que guarda el ledger.

Qué revisamos o cambiamos: Verificamos cada flecha contra el código del módulo de pagos. Corregimos un paso donde el diagrama mostraba que el reverso lo iniciaba el usuario, cuando en realidad lo dispara el núcleo al detectar timeout o diferencia.

Entrada 6 - Especificación OpenAPI 3 y mapeo de mensajería

Fecha: 05/11/2026 **Entregable:** Especificación de API y mapeo ISO 8583 / ISO 20022

Prompt del equipo:

Necesitamos la especificación OpenAPI 3 de nuestra API y un mapeo de mensajería. No tenemos claro si nos toca ISO 8583 o ISO 20022. ¿Nos explicas la diferencia como para estudiantes y hacemos el mapeo de nuestros mensajes?

Qué hizo la IA: Explicó que ISO 8583 es el estándar de tarjetas y ISO 20022 el de transferencias y reportes, y mapeó nuestros mensajes a ambos: pacs.008/pacs.002 para la transferencia, camt.056/pacs.004 para la devolución y camt.053/camt.054 para la conciliación. Generó la especificación a partir de docs/contrato-api.md.

Qué revisamos o cambiamos: Lo pasamos a Word con nuestro formato y añadimos las referencias bibliográficas. Validamos la especificación en un editor de OpenAPI para asegurar que no tuviera errores de sintaxis.

Entrada 7 - Plan de certificación

Fecha: 06/11/2026 **Entregable:** Plan de certificación (certificaciones, actores, casos de prueba)

Prompt del equipo:

¿Qué certificaciones necesitaría Ahorra+ para salir a producción en Venezuela? Ayúdanos con un plan que diga qué certificaciones, quiénes participan (SUDEBAN, bancos, el suiche, nosotros) y los casos de prueba. No queremos inventar normas, si no estás seguro de algo dínoslo.

Qué hizo la IA: Propuso la estructura del plan: certificaciones requeridas, matriz de actores y responsabilidades, y casos de prueba con resultado esperado (aprobación, rechazo, timeout, reverso, conciliación con diferencia, KYC rechazado por listas de sanciones).

Qué revisamos o cambiamos: Revisamos con fuentes oficiales las normas citadas y quitamos lo que no pudimos confirmar. Los casos de prueba los enlazamos con los escenarios del simulador para poder ejecutarlos de verdad.

Entrada 8 - Tablero de KPIs

Fecha: 05/11/2026 **Entregable:** Tablero de KPIs del prototipo

Prompt del equipo:

Hay que medir en el prototipo la tasa de aprobación, la latencia p95, los TPS y la tasa de reversos. ¿Nos haces un tablero sencillo que lo muestre? Y explícanos qué es el p95, porque no lo tenemos claro.

Qué hizo la IA: Explicó el p95 como “el 95% de los mensajes respondió en este tiempo o menos”. Creó tablero-kpis/ (HTML + JS sin dependencias) con modo demo y modo en vivo contra GET /api/v1/backoffice/kpis, y las fórmulas en CalculadoraKpis.java para que demo y vivo midan exactamente lo mismo.

Qué revisamos o cambiamos: Hicimos una ráfaga de pagos con el simulador y comparamos a mano unos cuantos valores con los del tablero. Pedimos que los timeouts se contaran aparte para no inflar la latencia.

Entrada 9 - Modelo de IA baseline y dataset sintético

Fecha: 06/11/2026 **Entregable:** Primera versión del modelo de IA (baseline) y su dataset

Prompt del equipo:

Para la parte de IA queremos una primera versión sencilla (baseline) que clasifique el perfil del usuario y le sugiera una regla de ahorro. No tenemos datos reales, ¿cómo armamos un dataset sintético y lo documentamos bien? Y que no use datos personales.

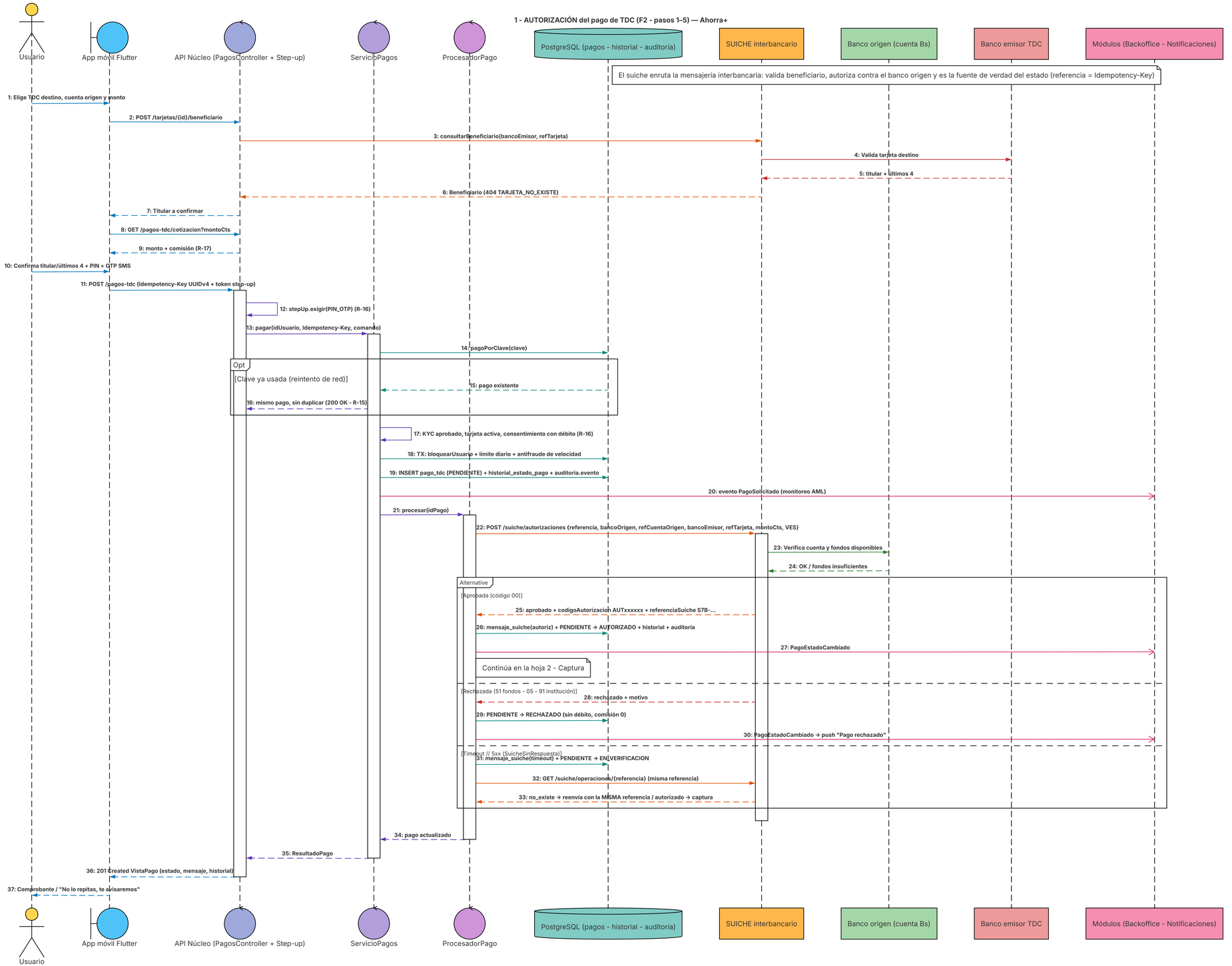
Qué hizo la IA: Propuso un baseline por reglas versionadas (ClasificadorPerfil, “reglas-1.2”) que usa solo rasgos seudonimizados: rango de ingreso (no el monto), volatilidad del saldo, objetivo, horizonte y tolerancia al riesgo. Sugirió documentar el dataset sintético con sus variables, rangos, cómo se generó y sus limitaciones.

Qué revisamos o cambiamos: Dejamos claro que el baseline es un punto de partida para comparar modelos futuros. El asistente que redacta los consejos agrega el aviso de “no es asesoría financiera”.

Declaración de uso responsable

La IA se usó como apoyo para diseñar, programar y documentar. Las decisiones de arquitectura, la revisión del código, la validación de las normas y el contenido final de los entregables son responsabilidad del equipo. No se compartieron datos personales reales: todos los usuarios, cuentas y pagos del prototipo son sintéticos o simulados.

1 - AUTORIZACIÓN del pago de TDC (F2 - pasos 1–5) — Ahorra+



LEYENDA - Actores y su rol en la AUTORIZACIÓN

- Usuario — Titular de la TDC. Elige monto/cuenta, confirma titular y últimos 4, y firma con PIN + OTP SMS (step-up PIN_OTP).
- App móvil Flutter — Genera la Idempotency-Key (UUIDv4) UNA vez por intento y la reutiliza en cada reintento de red; muestra estado y mensaje.
- API Núcleo (PagosController) — Adaptador REST Spring Boot del monolito modular: valida JWT, exige step-up PIN_OTP y la cabecera Idempotency-Key.
- ServicioPagos — Caso de uso (puerto de entrada): idempotencia R-15, KYC, tarjeta activa, consentimiento con débito R-16, límite diario y antifraude.
- ProcesadorPago — Máquina de estados R-14 contra el suiche: cada transición con bloqueo de fila, historial, auditoría y evento de dominio.
- PostgreSQL — pagos.pago_tdc - historial_estado_pago - mensaje_suiche - auditoria.evento (registro inmutable).
- SUICHE interbancario — Red que enruta la mensajería entre bancos: valida el beneficiario con el emisor, AUTORIZA contra el banco origen (retiene fondos), es idempotente por referencia y es la fuente de verdad del estado. Ahorra+ nunca custodia fondos.
- Banco origen — Mantiene la cuenta en Bs del usuario: valida existencia y fondos; aprueba o rechaza (51 fondos insuficientes).
- Banco emisor TDC — Emisor de la tarjeta destino: confirma titular y últimos 4 dígitos (nunca PAN).
- Módulos (eventos) — Backoffice: monitoreo AML de PagoSolicitado. Notificaciones: push por PagoEstadoCambiado.

LEYENDA - Código de flechas

- Usuario ↔ App ↔ API (HTTPS REST). Punteada = respuesta
- Llamada interna entre capas (puertos/casos de uso)
- Persistencia en PostgreSQL (transacción)
- Mensaje Ahorra+ ↔ Suiche (HTTP con timeout)
- Suiche ↔ Banco origen (cuenta Bs)
- Suiche ↔ Banco emisor TDC / rechazo
- Evento de dominio asíncrono (Spring Modulith)
- Línea punteada = respuesta / retorno

2 - CAPTURA del pago de TDC (F2 - pasos 6–7) — Ahorra+

Usuario
App móvil Flutter
API Núcleo (PagosController)
Worker de pagos (cada 30 s)
ProcesadorPago
PostgreSQL (pagos - historial - auditoria)
SUICHE interbancario
Banco origen (cuenta Bs)
Banco emisor TDC
Notificaciones
Puntos (lealtad)

Precondición: pago AUTORIZADO con codigoAutorizacion y referenciaSuiche (hoja 1)

1: POST /suiche/capturas {referencia + Idempotency-Key, codigoAutorizacion}
2: Valida código de autorización y estado (idempotente por referencia)

Alternative

[Captura aprobada (00)]

3: Débito de la cuenta origen en Bs
4: Débito aplicado
5: Abono al saldo de la TDC
6: Abono aplicado
7: aprobado 00 + referenciaSuiche
8: mensaje_suiche(captura) + AUTORIZADO → CAPTURADO + fecha_hora_captura
9: esPuntual = captura ≤ fecha límite del estado de cuenta (R-18)
10: historial_estado_pago + auditoria.evento (actor suiche)
11: PagoEstadoCambiado (CAPTURADO)
12: Push "Pago aplicado a tu TDC **1234"
13: PagoCapturado (esPuntual, alianza del banco emisor)
14: Acredita puntos del banco aliado si fue puntual (idempotente "pago:id")

[Captura rechazada por el emisor (05)]

15: rechazado 05 "Operación no permitida por el emisor"
16: POST /suiche/reversos (libera la autorización - ver hoja 3)
17: reverso aprobado (sin movimiento de fondos)
18: AUTORIZADO → RECHAZADO (sin débito, comisión 0 - R-17)
19: PagoEstadoCambiado (RECHAZADO)
20: Push "Pago rechazado: no se debitó nada"

[Timeout // 5xx del suiche (resultado desconocido)]

21: mensaje_suiche(timeout) + AUTORIZADO → EN_VERIFICACION
22: PagoEstadoCambiado (EN_VERIFICACION)
23: Push "Estamos confirmando tu pago. No lo repitas."

Loop

[Worker cada 30 s (también AUTORIZADOS huérfanos > 2 min)]

24: resolverEnVerificacion() → verificar(pago)
25: GET /suiche/operaciones/{referencia} (nunca reenvía a ciegas)
26: estado: capturado / autorizado / rechazado / reversado

Alternative

[capturado]

27: EN_VERIFICACION → CAPTURADO (+ eventos de notificación y puntos)

[autorizado]

28: Reintenta POST /suiche/capturas con la misma referencia

[rechazado // reversado]

29: EN_VERIFICACION → RECHAZADO / REVERSADO

30: Abre el detalle del pago
31: GET /pagos-tdc/{id}
32: pago_tdc + historial_estado_pago
33: estado actual + historial
34: VistaPago (CAPTURADO, referenciaSuiche, esPuntual, historial)
35: "Pago aplicado. ¡Pagaste a tiempo!"

Usuario
App móvil Flutter
API Núcleo (PagosController)
Worker de pagos (cada 30 s)
ProcesadorPago
PostgreSQL (pagos - historial - auditoria)
SUICHE interbancario
Banco origen (cuenta Bs)
Banco emisor TDC
Notificaciones
Puntos (lealtad)

LEYENDA - Actores y su rol en la CAPTURA

- Usuario — Recibe el push del resultado y consulta el detalle del pago; no debe repetir un pago en verificación.
- App móvil Flutter — Consulta GET /pagos-tdc/{id} y muestra estado, referencia del suiche, puntualidad e historial.
- API Núcleo — Adaptador REST de solo lectura del pago (vista con historial de estados).
- Worker de pagos — Programador (@Scheduled): resuelve pagos EN_VERIFICACION y AUTORIZADOS huérfanos consultando al suiche.
- ProcesadorPago — Envía la captura con la MISMA referencia; aplica AUTORIZADO → CAPTURADO, calcula puntualidad (R-18) y publica eventos.
- PostgreSQL — pago_tdc - historial_estado_pago - mensaje_suiche - auditoria.evento - puntos.
- SUICHE interbancario — Ejecuta la CAPTURA: ordena el débito en el banco origen y el abono en el emisor de la TDC. Es idempotente y la fuente de verdad ante timeouts (consulta por referencia).
- Banco origen — Debita la cuenta en Bs del usuario por orden del suiche (el dinero nunca pasa por Ahorra+).
- Banco emisor TDC — Abona el pago al saldo de la tarjeta; puede rechazar la captura (05) → reverso.
- Notificaciones — Push por cada PagoEstadoCambiado (aplicado, rechazado, en verificación).
- Puntos (lealtad) — Escucha PagoCapturado: si es puntual acredita puntos financiados por el banco aliado.

LEYENDA - Código de flechas

- Usuario ↔ App ↔ API (HTTPS REST)
- Llamada interna / programador
- Persistencia en PostgreSQL
- Mensaje Ahorra+ ↔ Suiche
- Suiche ↔ Banco origen (débito)
- Suiche ↔ Banco emisor (abono) / rechazo
- Evento de dominio asíncrono / push
- Punteada = respuesta / retorno

3 - REVERSO del pago de TDC — Ahorra+

Usuario
Analista backoffice
Backoffice (casos)
ServicioConciliacion
ProcesadorPago
PostgreSQL (pagos - historial - auditoría)
SUICHE interbancario
Banco origen (cuenta Bs)
Banco emisor TDC
Notificaciones

Disparadores: (a) el emisor rechaza la captura tras autorizar - (b) diferencia en conciliación (CAPTURADO → CON_DIFERENCIA) - (c) la consulta al suiche ya informa "reversado"

Alternative
[(a) Captura rechazada por el emisor]
1: reversarEnSuiche(pago, "Captura rechazada por el emisor")
[(b) Diferencia detectada en la conciliación]
2: transicion CAPTURADO → CON_DIFERENCIA
3: historial_estado_pago + auditoria.evento
4: reversarEnSuiche(pago, "Diferencia en conciliación: dif_monto / falta_suiche")
[(c) Consulta de estado devuelve "reversado"]
5: → REVERSADO directamente (sin enviar reverso)

6: POST /suiche/reversos {referencia = Idempotency-Key, motivo}

Alternative
[Operación CAPTURADA en el suiche]
7: Reintegro del débito a la cuenta origen (Bs)
8: Reintegro aplicado
9: Revierte el abono a la TDC
10: Abono revertido
11: aprobado 00 (operación reversada)
12: mensaje_suiche(reverso, aprobado)
13: CON_DIFERENCIA → REVERSADO + historial (actor suiche) + auditoría - comisión cobrada = 0 (R-17)
14: PagoEstadoCambiado (REVERSADO)
15: Push "Tu pago fue reversado; el dinero vuelve a tu cuenta"
[Operación solo AUTORIZADA (no capturada)]
16: Libera la autorización sin mover fondos (se liquida el día del reverso)
17: aprobado 00
18: mensaje_suiche(reverso) + AUTORIZADO → RECHAZADO (sin débito ni comisión)
19: PagoEstadoCambiado (RECHAZADO)
20: Push "Pago rechazado: no se debitó nada"
[No admite reverso (RECHAZADO / REVERSADO) o timeout]
21: rechazado 05 / sin respuesta
22: mensaje_suiche(reverso, rechazado | timeout) → el pago se mantiene CON_DIFERENCIA

Idempotente: repetir el reverso con la misma referencia devuelve la misma respuesta y nunca reintegra dos veces

Opt
[Seguimiento manual (pago CON_DIFERENCIA)]
23: DiferenciaConciliacion → abre caso_soporte (prioridad alta)
24: INSERT backoffice.caso_soporte
25: GET /backoffice/pagos?estado=CON_DIFERENCIA - GET /backoffice/casos
26: Pagos, referencia del suiche y diferencia en Bs
27: PATCH /backoffice/casos/{id} (resolución con el suiche / banco)
28: caso_soporte actualizado + auditoria.evento (actor operador)

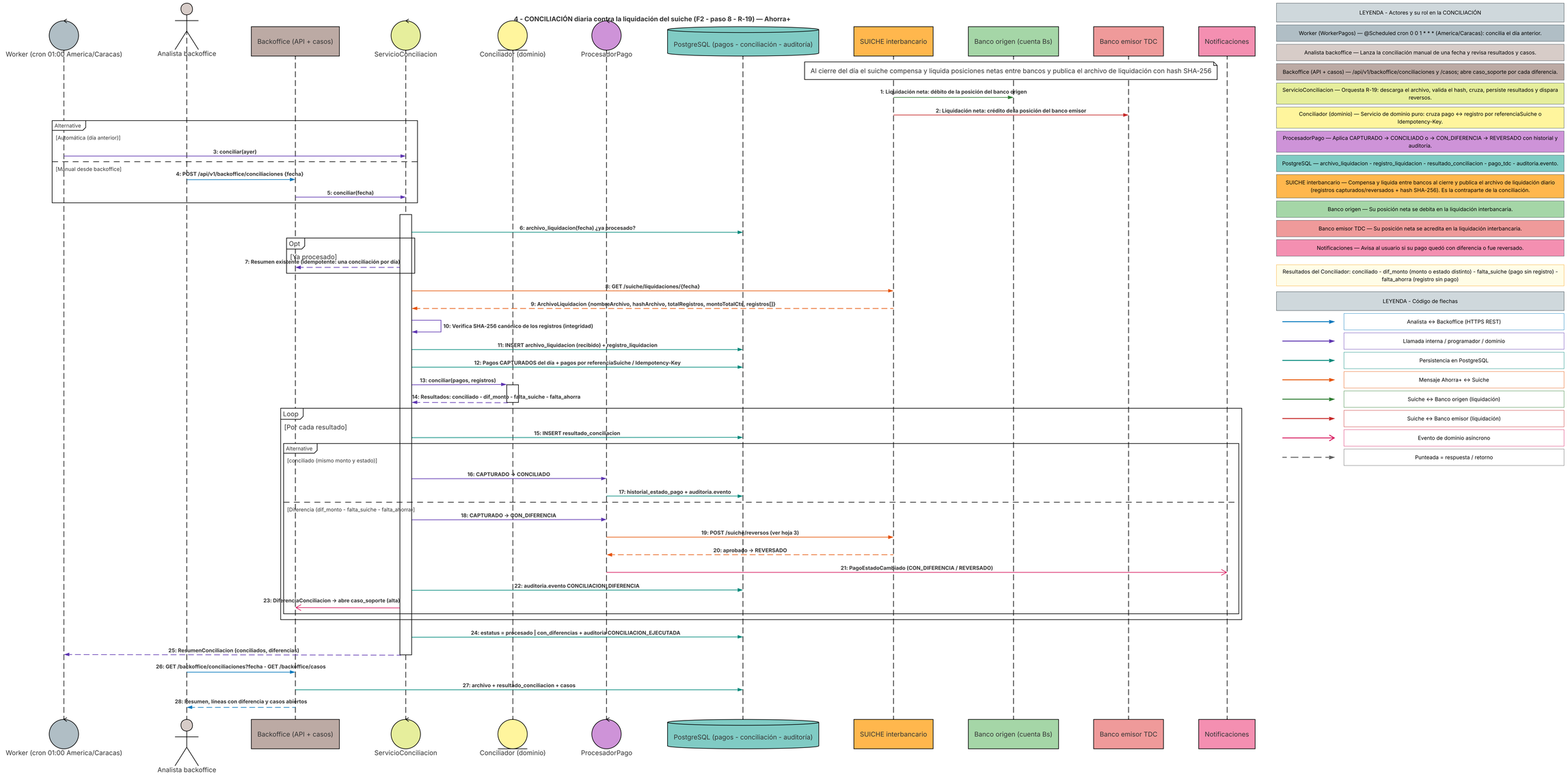
Usuario
Analista backoffice
Backoffice (casos)
ServicioConciliacion
ProcesadorPago
PostgreSQL (pagos - historial - auditoría)
SUICHE interbancario
Banco origen (cuenta Bs)
Banco emisor TDC
Notificaciones

LEYENDA - Actores y su rol en el REVERSO

- Usuario — Recibe el aviso de reverso: el monto vuelve a su cuenta de origen y no se cobra comisión.
- Analista backoffice — Operador de cumplimiento: revisa pagos CON_DIFERENCIA y resuelve el caso con el suiche/banco.
- Backoffice (casos) — Módulo que abre el caso_soporte por DiferenciaConciliacion y registra su resolución.
- ServicioConciliacion — Detecta la diferencia contra la liquidación y ordena CON_DIFERENCIA + reverso.
- ProcesadorPago — Solicita el reverso con la MISMA referencia y aplica la transición final (REVERSADO o RECHAZADO).
- PostgreSQL — mensaje_suiche (traza del reverso) - historial_estado_pago - auditoria.evento - backoffice.caso_soporte.
- SUICHE interbancario — Ejecuta el REVERSO: si la operación estaba capturada ordena el reintegro al banco origen y la reversión del abono en el emisor; si solo estaba autorizada, libera la autorización. Idempotente por referencia.
- Banco origen — Reintegra a la cuenta en Bs el monto debitado.
- Banco emisor TDC — Revierte el abono aplicado al saldo de la tarjeta.
- Notificaciones — Push al usuario por PagoEstadoCambiado (REVERSADO / RECHAZADO).

LEYENDA - Código de flechas

- Analista ↔ Backoffice (HTTPS REST)
- Llamada interna entre servicios
- Persistencia en PostgreSQL
- Mensaje Ahorra+ ↔ Suiche
- Suiche ↔ Banco origen (reintegro)
- Suiche ↔ Banco emisor / rechazo
- Evento de dominio asíncrono / push
- Punteada = respuesta / retorno


4 - CONCILIACIÓN diaria contra la liquidación del suiche (F2 - paso 8 - R-19) — Ahorra+
Worker (cron 01:00 America/Caracas)
Analista backoffice
Backoffice (API + casos)
ServicioConciliacion
Conciliador (dominio)
ProcesadorPago
PostgreSQL (pagos - conciliación - auditoría)
SUICHE interbancario
Banco origen (cuenta Bs)
Banco emisor TDC
Notificaciones
Al cierre del día el suiche compensa y liquida posiciones netas entre bancos y publica el archivo de liquidación con hash SHA-256
1: Liquidación neta: débito de la posición del banco origen
2: Liquidación neta: crédito de la posición del banco emisor
Alternative
[Automática (día anterior)]
3: conciliar(ayer)
[Manual desde backoffice]
4: POST /api/v1/backoffice/conciliaciones {fecha}
5: conciliar(fecha)
6: archivo_liquidacion(fecha) ¿ya procesado?
Opt
[Ya procesado]
7: Resumen existente (idempotente: una conciliación por día)
8: GET /suiche/liquidaciones/{fecha}
9: ArchivoLiquidacion (nombreArchivo, hashArchivo, totalRegistros, montoTotalCtd, registros[])
10: Verifica SHA-256 canónico de los registros (integridad)
11: INSERT archivo_liquidacion (recibido) + registro_liquidacion
12: Pagos CAPTURADOS del día + pagos por referenciaSuiche / Idempotency-Key
13: conciliar(pagos, registros)
14: Resultados: conciliado - dif_monto - falta_suiche - falta_ahorra
Loop
[Por cada resultado]
15: INSERT resultado_conciliacion
Alternative
[conciliado (mismo monto y estado)]
16: CAPTURADO → CONCILIADO
17: historial_estado_pago + auditoria.evento
[Diferencia (dif_monto - falta_suiche - falta_ahorra)]
18: CAPTURADO → CON_DIFERENCIA
19: POST /suiche/reversos (ver hoja 3)
20: aprobado → REVERSADO
21: PagoEstadoCambiado (CON_DIFERENCIA / REVERSADO)
22: auditoria.evento CONCILIACION_DIFERENCIA
23: DiferenciaConciliacion → abre caso_soporte (alta)
24: estatus = procesado | con_diferencias + auditoria CONCILIACION_EJECUTADA
25: ResumenConciliacion (conciliados, diferencias)
26: GET /backoffice/conciliaciones?fecha - GET /backoffice/casos
27: archivo + resultado_conciliacion + casos
28: Resumen, líneas con diferencia y casos abiertos
LEYENDA - Actores y su rol en la CONCILIACIÓN
Worker (WorkerPagos) — @Scheduled cron 0 0 1 * * * (America/Caracas): concilia el día anterior.
Analista backoffice — Lanza la conciliación manual de una fecha y revisa resultados y casos.
Backoffice (API + casos) — /api/v1/backoffice/conciliaciones y /casos; abre caso_soporte por cada diferencia.
ServicioConciliacion — Orquesta R-19: descarga el archivo, valida el hash, cruza, persiste resultados y dispara reversos.
Conciliador (dominio) — Servicio de dominio puro: cruza pago ↔ registro por referenciaSuiche o Idempotency-Key.
ProcesadorPago — Aplica CAPTURADO → CONCILIADO o → CON_DIFERENCIA → REVERSADO con historial y auditoría.
PostgreSQL — archivo_liquidacion - registro_liquidacion - resultado_conciliacion - pago_tdc - auditoria.evento.
SUICHE interbancario — Compensa y liquida entre bancos al cierre y publica el archivo de liquidación diario (registros capturados/reversados + hash SHA-256). Es la contraparte de la conciliación.
Banco origen — Su posición neta se debita en la liquidación interbancaria.
Banco emisor TDC — Su posición neta se acredita en la liquidación interbancaria.
Notificaciones — Avisa al usuario si su pago quedó con diferencia o fue reversado.
Resultados del Conciliador: conciliado - dif_monto (monto o estado distinto) - falta_suiche (pago sin registro) - falta_ahorra (registro sin pago)
LEYENDA - Código de flechas
Analista ↔ Backoffice (HTTPS REST)
Llamada interna / programador / dominio
Persistencia en PostgreSQL
Mensaje Ahorra+ ↔ Suiche
Suiche ↔ Banco origen (liquidación)
Suiche ↔ Banco emisor (liquidación)
Evento de dominio asíncrono
Punteada = respuesta / retorno